

SYSTEM DESIGN INTERVIEW

Design Google Drive / Dropbox

Cloud storage, block-level sync, and conflict resolution

~ 35 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement & Clarifying Questions
- 02 · Functional & Non-Functional Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Core Design Decisions
- 06 · Data Models & Schema
- 07 · File Upload Pipeline (Write Path)
- 08 · Storage Architecture & Deduplication
- 09 · Sync Protocol & Delta Detection
- 10 · File Download & Reconstruction

- [11](#) · Conflict Resolution
- [12](#) · Failure Modes & Recovery
- [13](#) · Trade-offs & Alternatives
- [14](#) · Interview Playbook

01

Problem Statement & Clarifying Questions

Define the scope of the file sync and cloud storage system

Design a cloud file storage and synchronization service like **Google Drive** or **Dropbox**. The system must store user files in the cloud, sync them across multiple devices, support sharing and version history, and minimise bandwidth via block-level deduplication.

Clarifying Questions

Question	Why It Matters	Typical Answer
Sync only or full storage?	Determines bandwidth model; sync implies local copies	Both: cloud storage + device sync
Concurrent users per account?	Conflict-resolution complexity; shared folders	Multiple devices per user; shared folders with collaborators
File size limits?	Drives chunking strategy and upload protocol	Google: 5 TB/file; Dropbox: 350 GB/file (older 2 GB)
Real-time sync SLA?	Notification latency, polling frequency, cost	Within 30 sec to 5 min acceptable
Offline support?	Local caching; conflict handling	Yes; sync when reconnected; detect conflicts
Sharing & permissions?	Metadata schema; access control	Yes: file/folder sharing, read/write/admin roles
Version history?	Storage and query complexity	Yes: keep last 30 versions (configurable)
Geographic distribution?	CDN, replication, latency	Global: US, EU, APAC; local edge caching

We focus on personal cloud storage with multi-device sync, block-level deduplication, and pragmatic conflict resolution. Real-time collaborative editing (Docs/Sheets) is out of scope — that is a CRDT/OT problem, not a file-sync problem.

02

Functional & Non-Functional Requirements

What the system must do, and how well

Functional Requirements

- **Upload files:** single or chunked multipart (>5 MB); resume on failure
- **Download files:** fetch from cloud or local cache; reconstruct from blocks
- **Sync daemon:** poll server for changes; detect local file changes
- **List files:** metadata API with pagination; folder hierarchy
- **Delete files:** soft delete with retention; hard delete after expiry
- **Share files / folders:** granular permissions (read, write, admin)
- **Version history:** keep previous versions; restore any version
- **Conflict resolution:** detect concurrent edits; LWW or versioning fallback
- **Offline support:** continue editing offline; sync on reconnect

Non-Functional Requirements

Requirement	Target
Availability	99.99% uptime SLA; graceful degradation if sync service fails
Latency	Sync detection <5 min; upload <500ms (metadata); download <1s (metadata)
Scale	1B+ users; 1 EB+ storage ; 100K+ concurrent active syncs; 1M+ RPS
Bandwidth	Minimise data transfer via block-level deduplication and delta sync
Storage cost	Tiered (S3 Standard / Glacier); per-user quota enforcement
Security	AES-256 at rest; TLS in transit; OAuth 2.0; audit logs
Consistency	Eventual globally; strong for metadata within a datacenter

03

Capacity Estimation

Math for scale

User & Storage

- **1B+ users** worldwide; ~30% active monthly → **~500M MAU**
- Avg storage per user: **~2 GB blended** (paid tiers ~10 GB, free tiers <1 GB)
- Total logical storage: 500M users × 2 GB ≈ **1 EB** (exabyte)
- Deduplication factor: ~40% (backups, duplicate documents, shared media)
- Net physical storage on S3: 1 EB × 0.6 ≈ **~600 PB**

WARNING

Why not 10 GB/user × 500M = 5 EB?

An earlier version of this guide assumed every user fills the free quota. In reality the distribution is heavily skewed: most accounts are free-tier with <1 GB used. A **2 GB blended average** is closer to Google/Dropbox public figures and yields ~1 EB logical (not 5 EB).

Sync Activity

- Peak concurrent active syncs: **100K–500K** (users actively editing/uploading)
- Sync frequency: ~5 min average; desktop daemon checks every 30 sec when foregrounded
- Files changed per user per day: avg 2–5 → **~100M file ops/day**
- Baseline bandwidth: 500M users × 100 MB/month ÷ 30 days ≈ **1.7 Gbps**
- Peak upload bandwidth: 100K concurrent × 1 Mbps = **100 Gbps** (not Tbps)

WARNING

Unit-check: Gbps vs Tbps

100,000 × 1 Mbps = 100,000 Mbps = **100 Gbps**. An earlier version printed 100 Tbps — off by a factor of 1,000. 100 Gbps is what a single AWS Direct Connect bundle delivers; 100 Tbps would be roughly all of Netflix's global egress at peak. Always sanity-check bandwidth answers against an anchor you trust.

RPS Estimates

- Upload RPS: 100K concurrent × 1 req/min = **~1.7K RPS** baseline; **~10K RPS** peak
- Download RPS: ~5K RPS (viewing, sharing downloads)
- Metadata RPS: list / get-versions / permission checks ≈ **~50K RPS**
- Sync delta queries: 100K × 1 query/5 min ≈ ~333 RPS baseline; spike to ~5K RPS

- Total API: **~60K RPS baseline, ~100K RPS peak**

KEY

Key Numbers

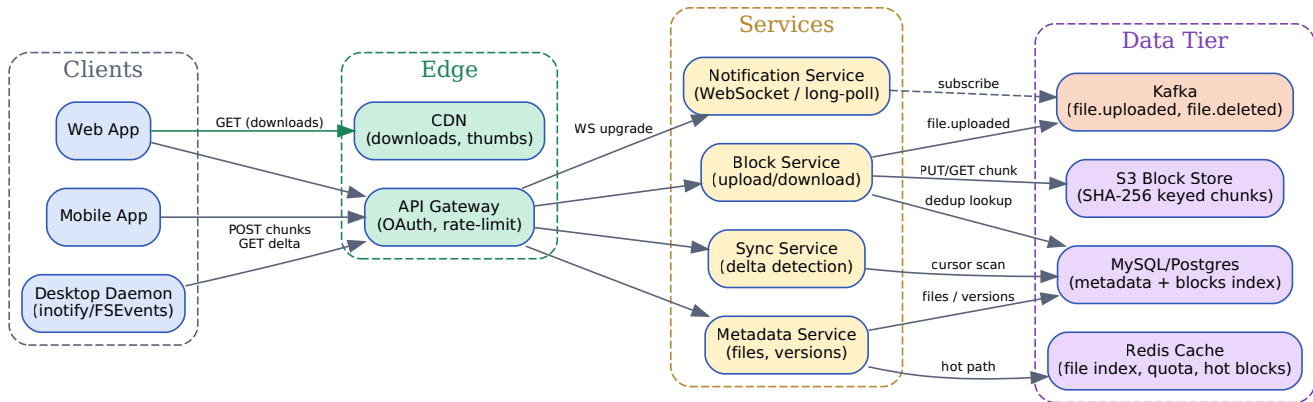
1 EB logical · ~600 PB physical · ~100 Gbps peak upload · 100K concurrent syncs · ~100K RPS at peak · 4 MB block size · SHA-256 dedup.

04

High-Level Architecture

System overview

A distributed system serving files with block-level sync, deduplication, and multi-device consistency. Reads are CDN-fronted; writes are chunked, hashed, and deduped before they ever touch S3.



End-to-end architecture: clients hit the CDN/API gateway, which fans out to Block, Metadata, and Notification services backed by S3, MySQL, Redis, and Kafka.

Tier Responsibilities

- **Client tier:** web (browser), desktop daemon (inotify/FSEvents/USN Journal), mobile (background sync, selective sync)
- **API gateway:** OAuth 2.0, MFA, per-user/IP rate limits, validation, circuit breakers
- **Block Service:** receives chunks, validates SHA-256, dedups against block index, writes new blocks to S3
- **Metadata Service:** file/folder records, permissions, versions; strongly consistent within region
- **Sync Service:** cursor-based delta detection; returns changed file list and block indices
- **Notification Service:** WebSocket push (with long-poll fallback); fans out file.uploaded events
- **S3 Block Store:** 4 MB content-addressed chunks; lifecycle into Glacier for cold blocks
- **MySQL/Postgres:** metadata (files, versions, file_blocks, blocks index)
- **Redis:** path → file_id lookups, quota counters, hot-block cache, session state (~99% hit target)
- **Kafka:** decouples upload from thumbnail generation, dedup verification, audit; topics file.uploaded / file.deleted / file.shared

05

Core Design Decisions

Block-level sync, dedup, delta protocol, conflict resolution

Block-Level Sync vs Full-File Sync

- **Chosen:** 4 MB content-addressed blocks; only changed blocks re-uploaded
- **Example:** 1 GB doc, edit one paragraph → upload one 4 MB block, not 1 GB
- **Bandwidth savings:** typically **10–50×** for incremental edits on large files
- **Trade-off:** tracks block versions, manifest reconstruction, dedup logic; worth it for the bandwidth

INSIGHT

Why 4 MB?

Smaller blocks (e.g. 1 MB) bloat the metadata index — a 1 GB file is 1,000 manifest entries. Larger blocks (e.g. 64 MB) re-upload too much on small edits. **4 MB** is Dropbox's empirical sweet spot: ~250 manifest rows per GB, ~6 round-trips for the median 25 MB file at 4 parallel uploads, and most office-doc edits land within a single block.

Deduplication Strategy

- **Content-addressed:** block key = SHA-256(block_data); identical blocks stored once
- **Dedup ratio:** typical **40–50%** (backups, shared documents, common media)
- **Cross-user dedup:** same file uploaded by N users → 1 physical block (privacy-careful: no metadata correlation)
- **Hash collisions:** birthday probability $\sim 10^{-29}$ per 600 PB store — vanishingly small, but verify on read

Delta Sync Protocol

- **Cursor:** client persists a monotonic sequence number (e.g. last-seen version_id)
- **Server query:** returns files modified since cursor (up to 1,000/page)
- **Block-level delta:** for files >10 MB, return only changed block indices, not full contents
- **Latency:** <50 ms metadata round-trip; bandwidth scales with diff, not file size

Conflict Resolution Strategy

- **Last-Write-Wins (LWW):** default; newer mtime wins
- **Versioning fallback:** if mtimes equal but content hashes differ, keep server version, rename local as {name}.conflict.{ts}
- **Premium versioning:** all versions retained; restore any version

- **Detection:** server tracks version vector; client reports local mtime + content hash

06

Data Models & Schema

Metadata in Postgres, blocks in S3

Metadata lives in PostgreSQL for ACID guarantees and joins; block data lives in S3 keyed by content hash. The `file_blocks` table is the manifest that ties a logical file version to its ordered list of blocks.

PostgreSQL Schema

```
CREATE TABLE files (  
  file_id          UUID PRIMARY KEY,  
  owner_id         BIGINT NOT NULL,  
  name             VARCHAR(255) NOT NULL,  
  path             VARCHAR(2048) NOT NULL,  
  mime_type        VARCHAR(100),  
  size_bytes       BIGINT,  
  created_at       TIMESTAMP,  
  modified_at      TIMESTAMP,  
  deleted_at       TIMESTAMP,          -- soft delete  
  current_version_id UUID,  
  UNIQUE (owner_id, path)  
);  
  
CREATE TABLE blocks (  
  block_hash       CHAR(64) PRIMARY KEY, -- SHA-256 hex  
  size_bytes       INT,  
  s3_key           VARCHAR(255) NOT NULL,  
  created_at       TIMESTAMP,  
  last_access_at   TIMESTAMP,  
  ref_count        INT DEFAULT 1         -- live references for GC  
);  
  
CREATE TABLE file_blocks (  
  file_version_id  UUID NOT NULL,  
  block_sequence   INT NOT NULL,  
  block_hash       CHAR(64) NOT NULL REFERENCES blocks(block_hash),  
  PRIMARY KEY (file_version_id, block_sequence)  
);  
  
CREATE TABLE versions (  
  version_id       UUID PRIMARY KEY,  
  file_id          UUID NOT NULL REFERENCES files(file_id),  
  created_at       TIMESTAMP,  
  modified_at      TIMESTAMP  
);  
  
CREATE TABLE user_quota (  
  user_id          VARCHAR(255) PRIMARY KEY,  
  quota_bytes       BIGINT NOT NULL,  
  used_bytes        BIGINT NOT NULL,  
  created_at       TIMESTAMP,  
  modified_at      TIMESTAMP  
);
```

```
user_id      BIGINT PRIMARY KEY,  
quota_bytes  BIGINT DEFAULT 16106127360, -- 15 GiB  
used_bytes   BIGINT DEFAULT 0  
);
```

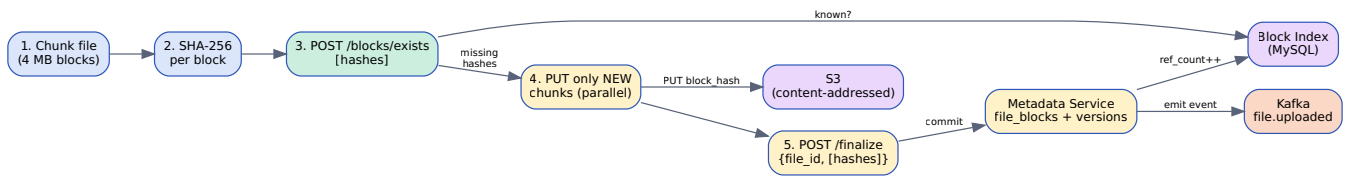
Key Design Decisions

- **Content addressing (block_hash):** block stored once per unique hash → cross-user dedup for free
- **Version history:** separate versions table; each version points to a manifest in file_blocks
- **Soft delete:** deleted_at; cleanup job hard-deletes after 30-day retention
- **Indexing:** (owner_id, path) unique for path lookups; block_hash PK for dedup probes
- **ref_count on blocks:** drives garbage collection — block is deletable when count drops to 0 for >30 days

07

File Upload Pipeline (Write Path)

Chunked multipart with block dedup and async fan-out



Block-level sync: chunk client-side, hash each chunk, ask the server which hashes it has not seen, upload only the new ones, then commit the manifest.

Small File (<5 MB)

- **Protocol:** single PUT /files/{file_id}
- **Flow:** Client → API Gateway → Block Service → S3 (direct put) → 200 OK
- **Latency:** <100 ms metadata; total time dominated by network upload

Large File (>5 MB)

1. Client chunks file into **4 MB blocks**
2. Client computes **SHA-256** for each block
3. Client calls POST /blocks/exists with hash list — server returns the subset it doesn't have
4. Client **PUTs only missing chunks**, up to 4 in parallel; ETag per part
5. Client calls POST /files/{file_id}/finalize with the ordered hash list (the manifest)
6. Server creates a new versions row, writes file_blocks, increments ref_count
7. Server publishes file.uploaded to Kafka → notification fan-out

TIP

Round-trip math

Median S3 PUT round-trip from a same-region service is ~30–60 ms (TLS + request + ack). Cross-region PUT can be 200–400 ms. With 4 parallel uploads a 25 MB file (≈7 blocks) needs ~2 round-trips of work — well under the <500 ms metadata target. The dedup probe avoids the round-trip entirely for unchanged blocks.

Async Fan-out

- **Block Processor:** consumes file.uploaded, verifies SHA-256, optionally compresses text blocks, runs orphan-block GC
- **Thumbnail Generator:** images/PDFs/Office docs → previews into a separate S3 bucket

- **Notification Service:** pushes change to other devices via WebSocket so they wake and fetch the delta

08

Storage Architecture & Deduplication

S3, content addressing, and garbage collection

S3 Layout

- **Key:** `s3://blocks-prod/{hash[:2]}/{hash[2:]}` — 256-way prefix sharding for hot-partition spreading
- **Block size:** 4 MB default; 100 KB for thumbnails; up to 10 MB for video
- **Storage classes:** S3 Standard for hot (<7 days); Intelligent-Tiering / Glacier for cold
- **Encryption:** SSE-S3 server-side; optional client-side (zero-knowledge) for paid tiers
- **Versioning:** S3 versioning *off* — version semantics live in our metadata DB

Deduplication Mechanics

- On upload, Block Service probes `blocks` table by SHA-256 hash
- Hit → bump `ref_count`, link new `file_blocks` row, no S3 write
- Miss → insert `blocks` row, PUT to S3 keyed by hash
- **Dedup ratio:** 40–50% measured; biggest wins are shared docs, OS backup files, and common media
- **Privacy:** dedup index is keyed by hash only; we never reveal who else has the block (counter increments are not logged per-user)

Garbage Collection

- Periodic job sums `ref_count` across all `file_blocks` referencing each hash
- If 0 for >30 days, delete S3 object and remove the `blocks` row
- Grace window prevents deleting a block whose only reference was a file mid-restore

Quota & Billing

- **Per-user quota:** Redis (hot) + Postgres (durable); enforced before upload
- **Billing model:** charge for *logical* bytes (user-visible), not physical — keeps the dedup win on the provider side and gives users predictable bills
- **Effective cost:** S3 Standard ~**2.3¢/GB-mo**, Glacier ~**0.4¢/GB-mo**; after 40% dedup, blended ~1.4¢/GB-mo of physical storage

09

Sync Protocol & Delta Detection

Cursor-based pull plus WebSocket push

Client-Side Sync Daemon

- **Polling interval:** 30 sec foreground / 5 min background
- **Local change detection:** inotify (Linux), FSEvents (macOS), USN Journal (Windows)
- **State store:** `.sync_state.db` (SQLite) — file metadata at last sync
- **Offline:** queue local changes; reconcile on reconnect; flag conflicts

Server Delta Detection

- **Cursor:** opaque token, internally a `(version_id, modified_at)` tuple
- **Query:** `SELECT ... FROM files WHERE owner_id = ? AND modified_at > cursor ORDER BY modified_at LIMIT 1000`
- **Response:** changed file list with `file_id`, `name`, `size`, `mtime`, `current_version_id`, `deleted` flag
- **Block-level delta:** for files >10 MB, also return changed block indices
- **Latency target:** <50 ms server query + network round-trip

Notification Service

- **WebSocket:** daemon holds a persistent connection; server pushes on file change
- **Fan-out:** Notification Service consumes `file.uploaded` from Kafka, looks up watching clients, pushes
- **Fallback:** if WebSocket fails, client falls back to HTTP long-polling (30 sec hold)
- **Push latency:** <1 sec end-to-end from upload-finalize to other-device notification

INSIGHT

Consistent hashing for the WebSocket fleet

Notification Service nodes hold millions of WebSocket connections. Use **consistent hashing** on `user_id` to route a user's devices to the same node, so a fan-out only sends one cross-node hop. Adding or draining a node moves only $\sim 1/N$ of users — critical when each rebalance tears down millions of connections.

10

File Download & Reconstruction

Manifest → parallel block fetch → stream

Small Files (single block)

- **Protocol:** GET /files/{file_id}
- **Flow:** Block Service fetches the single block from S3, streams to client
- **Caching:** Redis caches frequently accessed blocks; ~99% hit rate target on hot blocks

Large Files (multi-block)

1. Block Service queries `file_blocks` for the file's current version
2. Block Service issues 4–8 parallel S3 GETs against the manifest
3. As blocks arrive, stream to client immediately (don't wait for all blocks)
4. Supports HTTP Range header for resumable downloads
5. Latency: <100 ms metadata; perceived TTFB ≈ first block round-trip (~30–60 ms in-region)

Selective and Smart Sync

- **Selective sync:** user picks folders to materialise on disk; others stay cloud-only
- **Smart sync:** metadata is always synced; file content fetches on access (placeholder file → real file on first read)
- **Conflict guard:** if local file changed mid-download, abort, requeue after conflict resolution

11

Conflict Resolution

Concurrent edits across devices

Detection

- **Trigger:** on upload-finalize, server compares client's reported parent version with `current_version_id`
- **Stale parent:** client edited an old version → conflict
- **Equal mtime, different hash:** two devices saved at the same second with different content → concurrent edit
- **Metadata conflicts:** both sides renamed the same file → resolve by path lineage

Last-Write-Wins (LWW)

- **Rule:** compare modification timestamps; newer wins
- **Pros:** simple, deterministic, no UX surprises in the common case
- **Cons:** silent data loss if both versions had real content; unsuitable for collaborative docs

Versioning Fallback

- **Action:** keep server version as authoritative; rename local copy as `{name}.conflict.{ts}.{ext}`
- **Example:** `Plan.docx` server v5 vs client v4 (modified) → save `Plan.conflict.1715000000.docx`, pull server v5
- **User action:** manually merge; delete the conflict copy when done
- **Pros:** no data loss; **Cons:** manual merge effort, especially on shared folders

When to Use OT / CRDT

- Real-time collaborative editing (Google Docs / Sheets) uses OT or CRDT, not file sync
- For binary files (images, ZIPs, videos) text-merge approaches don't apply
- Dropbox Smart Sync / Google Drive integrate with the editor app's API for live collaboration

TIP

Pragmatic default

Ship LWW with the conflict-rename fallback. It is <200 lines of code, covers ~99% of real-world conflicts (different devices, not different humans in the same second), and crucially has no data loss path even when it's wrong — the loser's content lands on disk as a renamed file.

12

Failure Modes & Recovery

What can go wrong, how we detect and recover

Failure	Detection	Recovery
S3 outage	Upload returns 5xx; download falls back to cache	Exponential backoff; cross-region S3 fallback bucket; retry queue
Metadata DB unavailable	Connection timeout; circuit breaker opens	Read replicas serve reads; queue writes locally; reconcile on recovery
Upload interrupted	Client detects TCP reset / 0 bytes ack	Resume from last confirmed block via ETag; manifest only commits when all blocks present
Sync daemon crash	No heartbeat; pending push notifications undelivered	Daemon auto-restarts; on launch performs full delta sync from cursor
Block hash collision	Two blocks with same hash but byte-different content ($\sim 10^{-29}$ probability)	Cryptographic re-validation on read; store as separate entries with disambiguating suffix
Quota exceeded	User hits limit; next upload rejected	Soft enforcement: block uploads, prompt to upgrade or delete; auto-delete oldest versions if user opts in
Concurrent delete + upload	File deleted on server while client uploading	Accept upload as a new file version; on next sync, client surfaces the un-delete
Shared folder permission revoked	Daemon gets 403 on next sync	Remove local copy; write audit-log entry; notify user
Notification Service node loss	WebSockets drop; clients get connection-closed	Clients reconnect with consistent-hash retry; long-poll fallback bridges the gap

WARNING

Graceful degradation order

When something is on fire, prioritise **downloads** (users must read their files), then **uploads**, then **sync notifications**, then **thumbnails / audit**. Never block the read path on a non-essential subsystem.

13

Trade-offs & Alternatives

Decisions and rationale

Decision	Alternative	Why Chosen
4 MB block size	1 MB (more granular) or 64 MB (fewer manifests)	Balances metadata overhead against re-upload cost; sweet spot for typical edit patterns
SHA-256 hash	MD5 (faster) or BLAKE3	Collision-resistant for adversarial inputs; CPU cost negligible with HW acceleration; widely standardised
LWW conflict resolution	Always-version, or CRDT/OT	Simplest UX for the 99% case; rename-on-conflict prevents data loss; CRDTs aren't useful for binary files
PostgreSQL metadata	DynamoDB or Cassandra	ACID for the file/version invariants; joins useful for sharing/quota queries; sharded by user_id when needed
S3 storage	Custom object store	Cost: S3 Standard ~2.3¢/GB-mo (Glacier ~0.4¢) vs custom ~1¢/GB-mo; custom requires major ops investment
Eventual consistency	Strong consistency globally	Availability and latency dominate; 5-minute sync window is acceptable; metadata is strongly consistent within a region
Kafka async fan-out	Synchronous fan-out	Decouples upload from thumbnails / dedup verification / audit; improves perceived upload latency
WebSocket + long-poll	Polling only (5-min interval)	Real-time UX matters; WebSocket is cheap once you have it; long-poll is the fallback when intermediaries break WS
Client-side chunking	Server-side chunking	Saves the entire file upload before dedup probe; server only sees blocks it doesn't already have

INSIGHT

Headline tension

The architecture trades **complexity** (block manifests, dedup index, version vectors, push fabric) for **bandwidth and storage cost**. At 1 EB logical storage and 100 Gbps peak upload, even a 40% reduction is worth a lot of engineering. At 10 GB total it would not be.

14

Interview Playbook

How to present this design (45 minutes)

A 45-minute interview pacing for Drive/Dropbox. Lead with the bandwidth story (block-level sync), defend the dedup numbers, and pick one or two deep dives — don't try to cover everything.

Opening (5 min): Clarify & Scope

- Ask 2–3 questions: sync + storage? multi-device? sharing? version history?
- Confirm scale: **1B users, ~1 EB logical, 100K concurrent syncs**
- Set scope: focus on core sync + upload/download; mention but don't deep-dive sharing/collab

Capacity (3 min)

- Storage: $500M \text{ MAU} \times 2 \text{ GB} \approx \mathbf{1 \text{ EB logical}}$; 40% dedup $\rightarrow \mathbf{\sim 600 \text{ PB physical}}$
- Sync: $100K \text{ concurrent} \times 1 \text{ query/5 min} \approx \sim 333 \text{ RPS baseline}$
- Bandwidth: $100K \text{ concurrent} \times 1 \text{ Mbps} = \mathbf{100 \text{ Gbps}}$ peak (Gbps, not Tbps!)
- Block-level delta: **10–50× bandwidth savings** vs full-file sync

Architecture (10 min)

- Client tier (web, daemon, mobile) $\rightarrow$ API gateway $\rightarrow$ Block / Metadata / Sync / Notification services
- Storage: S3 content-addressed blocks + Postgres metadata + Redis hot path
- Async: Kafka topics for fan-out (thumbnail, dedup verify, notification)
- Draw the diagram — interviewers care that you can decompose into services

Deep Dives (20 min): Pick 2

- **Upload & dedup:** chunk $\rightarrow$ hash $\rightarrow$ probe $\rightarrow$ upload missing $\rightarrow$ finalize manifest
- **Delta sync:** cursor-based query, block-level diffs, WebSocket push
- **Conflict resolution:** LWW + rename-fallback; when CRDT/OT applies
- **Storage architecture:** 4 MB blocks, SHA-256 keys, S3 prefix sharding, GC
- **Failure recovery:** resumable uploads, metadata replicas, partition tolerance

Wrap-up (5 min): Trade-offs

- Block-level sync vs full-file: complexity worth it for bandwidth at 1 EB scale
- Eventual consistency vs strong: acceptable for file sync; metadata strong within region
- Custom store vs S3: S3 wins on ops cost; $\sim 2.3\text{¢/GB-mo}$ Standard, 0.4¢ Glacier

- Closing line: *"Block-level dedup + delta sync are the differentiators that make this economical at exabyte scale."*

KEY

Numbers to memorise

1 B users / 500M MAU · 2 GB blended avg · 1 EB logical / ~600 PB physical · 4 MB blocks / SHA-256
· 40–50% dedup · 100K concurrent syncs · 100 Gbps peak upload · ~50 ms delta query · S3 ~2.3¢/
GB-mo Standard, ~0.4¢ Glacier · 99.99% availability.

What Interviewers Probe

- **Block-level thinking:** Why 4 MB? How is dedup verified? Reconstruction overhead?
- **Sync protocol:** How are changes detected? Offline devices? Conflicts?
- **Consistency model:** Why eventual? Edge cases for concurrent edits?
- **Scale intuition:** Defend every capacity number; unit-check bandwidth (Gbps vs Tbps)
- **Trade-offs:** Why not S3 versioning? Why custom daemon? Why 4 MB not 1 MB?

INSIGHT

Five takeaways

(1) Block-level sync is the differentiator — small edits trigger 4 MB transfers, not GB transfers, for ~10–50× bandwidth savings. **(2)** Content-addressed storage gives 40–50% dedup nearly for free. **(3)** Cursor-based delta sync + WebSocket push delivers <1 s cross-device propagation. **(4)** LWW with rename-fallback handles conflicts pragmatically without data loss. **(5)** Kafka decouples the user-facing upload latency from async work (dedup verify, thumbnails, audit).